

编译原理-语义分析 01

语义分析基础、语法制导翻译

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 4 月 24 日

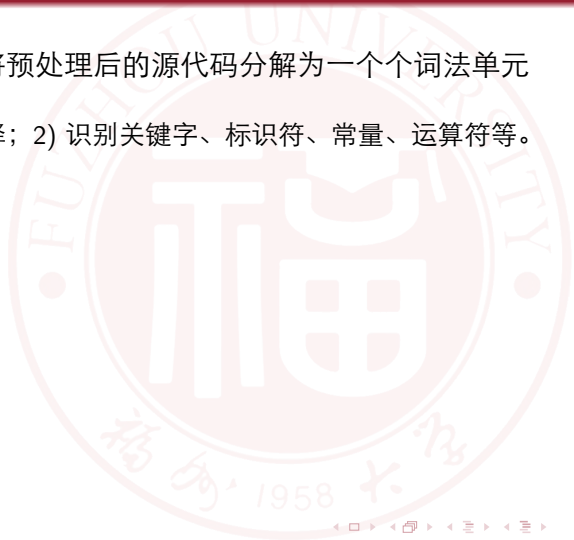


福州大学
FUZHOU UNIVERSITY 1958

- ① 知识点回顾
- ② 语义分析
- ③ 语法制导翻译

编译阶段

- ① 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
- 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。



编译阶段

- ① 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
 - 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。
- ② 语法分析 (Syntax Analysis): 根据语言的语法规则 (文法), 将词法单元序列解析为抽象语法树 (Abstract Syntax Tree, AST)。ul style="list-style-type: none;">- 过程: 检查语法是否正确, 构建语法树来表示程序的结构。
- 输出: 抽象语法树。

编译阶段

- ① 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
 - 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。
- ② 语法分析 (Syntax Analysis): 根据语言的语法规则 (文法), 将词法单元序列解析为抽象语法树 (Abstract Syntax Tree, AST)。
 - 过程: 检查语法是否正确, 构建语法树来表示程序的结构。
 - 输出: 抽象语法树。
- ③ 语义分析阶段 (Semantic Analysis): 检查程序的语义是否正确, 例如类型检查、变量作用域分析等。
 - 过程: 检查变量是否声明, 检查类型是否匹配, 构建符号表 (Symbol Table), 记录变量、函数等的定义和属性。
 - 经过语义检查和修饰的抽象语法树。

- ① 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
 - 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。
- ② 语法分析 (Syntax Analysis): 根据语言的语法规则 (文法), 将词法单元序列解析为抽象语法树 (Abstract Syntax Tree, AST)。
 - 过程: 检查语法是否正确, 构建语法树来表示程序的结构。
 - 输出: 抽象语法树。
- ③ 语义分析阶段 (Semantic Analysis): 检查程序的语义是否正确, 例如类型检查、变量作用域分析等。
 - 过程: 检查变量是否声明, 检查类型是否匹配, 构建符号表 (Symbol Table), 记录变量、函数等的定义和属性。
 - 经过语义检查和修饰的抽象语法树。

- 授课教师：柯嵩宇



① 知识点回顾

② 语义分析

语义分析概述

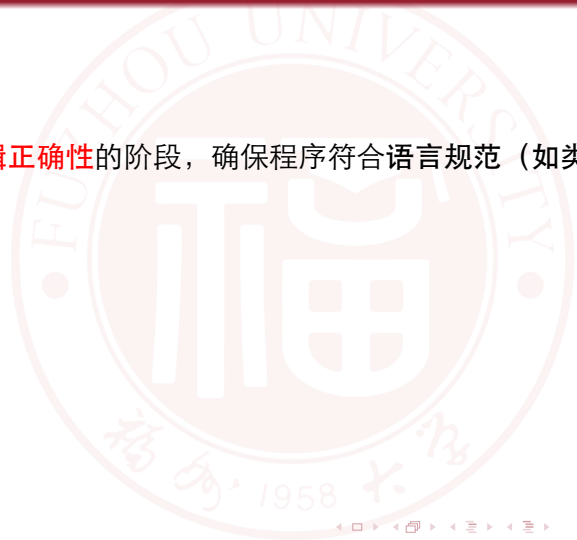
符号表管理

基于 ANTLR Listener 的语义分析

③ 语法制导翻译

语义分析的任务

语义分析是编译过程中**验证程序逻辑正确性**的阶段，确保程序符合语言规范（如类型匹配、作用域规则）。



语义分析的任务

语义分析是编译过程中**验证程序逻辑正确性**的阶段，确保程序符合语言规范（如类型匹配、作用域规则）。
这里给出一个语法合法但语义不合法的示例：

例

```
int x = "Hello World";
```

语义分析的三目标

- ① 静态语义检查；
- ② 构建符号表 (Symbol Table) ；
- ③ 为中间代码生成或优化提供语义信息；



语义分析的三目标

① 静态语义检查；

- 目的：在编译阶段识别不符合语言规范的逻辑错误，避免程序运行时崩溃或不可预测行为。
- 意义：弥补语法分析的局限性，通过语言语义规则¹ 验证程序结构的逻辑一致性，显著提升代码可靠性。它可在编译时拦截数据类型不匹配等错误，减少调试成本。

② 构建符号表 (Symbol Table) ；

③ 为中间代码生成或优化提供语义信息；

¹类型系统、作用域规则等

语义分析的三目标

① 静态语义检查；

- 目的：在编译阶段识别不符合语言规范的逻辑错误，避免程序运行时崩溃或不可预测行为。
- 意义：弥补语法分析的局限性，通过语言语义规则¹ 验证程序结构的逻辑一致性，显著提升代码可靠性。它可在编译时拦截数据类型不匹配等错误，减少调试成本。
- 例：a[3.14] 语法合法，但语义错误。

② 构建符号表 (Symbol Table) ；

③ 为中间代码生成或优化提供语义信息；

¹类型系统、作用域规则等

语义分析的三目标

① 静态语义检查；

- 目的：在编译阶段识别不符合语言规范的逻辑错误，避免程序运行时崩溃或不可预测行为。
- 意义：弥补语法分析的局限性，通过语言语义规则¹ 验证程序结构的逻辑一致性，显著提升代码可靠性。它可在编译时拦截数据类型不匹配等错误，减少调试成本。
- 例：a[3.14] 语法合法，但语义错误。

② 构建符号表 (Symbol Table) ；

- 目的：建立程序实体的结构化信息库，为跨模块的符号引用提供上下文支持。
- 意义：符号表是编译器管理标识符的核心数据结构，通过作用域嵌套和类型绑定，确保变量、函数等实体的唯一性和可见性。

③ 为中间代码生成或优化提供语义信息；

¹类型系统、作用域规则等

语言规范的逻辑错误，避免程序运行时崩溃或不
通过语言语义规则¹ 验证程序结构的逻辑一致性
编译时拦截数据类型不匹配等错误，减少调试成本
错误。

信息库，为跨模块的符号引用提供上下文支持。
符号的核心数据结构，通过作用域嵌套和类型绑定
和可见性。

信息；
类型生成不同的算术指令（如整数加法与浮点加
定变量的存储周期（如栈帧分配）。

- **目的**：在编译阶段识别不符合语言规范的逻辑错误，避免程序运行时崩溃或不可预测行为。
- **意义**：弥补语法分析的局限性，通过语言语义规则¹ 验证程序结构的逻辑一致性，显著提升代码可靠性。它可在编译时拦截数据类型不匹配等错误，减少调试成本。
- **例**：a[3.14] 语法合法，但语义错误。

- **目的：**建立程序实体的结构化信息库，为跨模块的符号引用提供上下文支持。
- **意义：**符号表是编译器管理标识符的核心数据结构，通过作用域嵌套和类型绑定，确保变量、函数等实体的唯一性和可见性。

- 例如，强类型语言需根据变量类型生成不同的算术指令（如整数加法与浮点加法指令不同），而作用域信息则决定变量的存储周期（如栈帧分配）。

语义分析示例

```
1 void func() {  
2     a = 5; // 变量 a 未在函数作用域或全局作用域声明  
3 }
```

```
1 String s = "value";  
2 int result = s + 5; // 语义分析拒绝字符串与整型的 + 操作
```

```
1 int x = 10;  
2 void func() {  
3     float x = 3.14; // 内层作用域覆盖外层 x  
4     printf("%f", x); // 符号表确定此处 x 为浮点型  
5 }
```

语义分析的核心任务

- ① 类型检查：验证表达式、赋值、函数调用等操作的合法性（如整型与浮点型混合运算的隐式转换）。
- ② 作用域分析：解析变量 / 函数的可见性（如局部变量覆盖全局变量）。
- ③ 一致性检查：函数参数数量与声明匹配、结构体成员访问合法性等。
- ④ 控制流验证：检测不可达代码、循环 / 条件语句的合理结构。
- ⑤ 符号表管理：动态维护标识符的语义信息（如类型、内存地址）。



① 知识点回顾

② 语义分析

语义分析概述

符号表管理

基于 ANTLR Listener 的语义分析

③ 语法制导翻译

核心结构与功能

符号表管理是编译过程中维护程序语义正确性的核心机制。其核心结构为**键值对 (Key-Value)**。



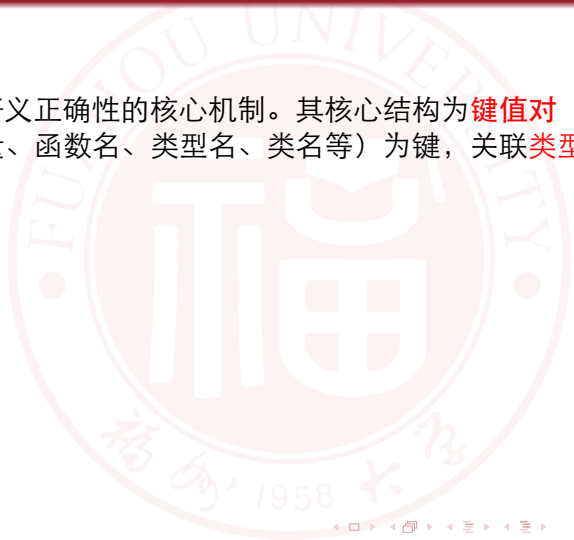
核心结构与功能

符号表管理是编译过程中维护程序语义正确性的核心机制。其核心结构为**键值对 (Key-Value)**。以标识符名称（变量、函数名、类型名、类名等）为键，



核心结构与功能

符号表管理是编译过程中维护程序语义正确性的核心机制。其核心结构为**键值对 (Key-Value)**。以标识符名称（变量、函数名、类型名、类名等）为键，关联**类型、作用域、内存地址等属性**。



核心结构与功能

符号表管理是编译过程中维护程序语义正确性的核心机制。其核心结构为**键值对 (Key-Value)**。以标识符名称（变量、函数名、类型名、类名等）为键，关联类型、作用域、内存地址等属性。

符号表的功能（即操作接口）：

- ① 插入（声明时注册符号）；
- ② 查询（引用时验证存在性）；
- ③ 删除（退出作用域时清理）。

特别地，这些操作都需要高效进行²。

²需要使用哈希表、二叉查找树、可持久化数据结构等

符号表管理综合示例

```
1  int global;           // 全局符号表: global → int
2  void main() {
3      int x = 0;         // 压入main局部符号表: x → int
4      {
5          char x = 'a';  // 压入块符号表, 覆盖外层x
6          global = x;    // 查询块内x(char)和全局global(int),
7                          // 触发隐式类型转换警告
8      }                 // 弹出块符号表, 恢复外层x(int)
9  }
```

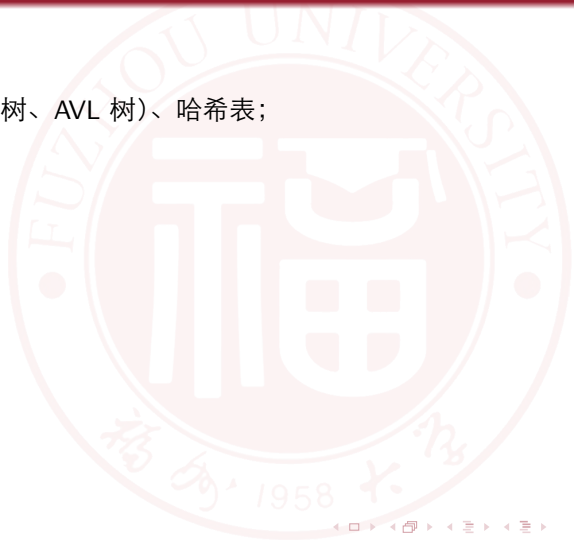
符号表的实现指导

- 如何管理键值对?



符号表的实现指导

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；



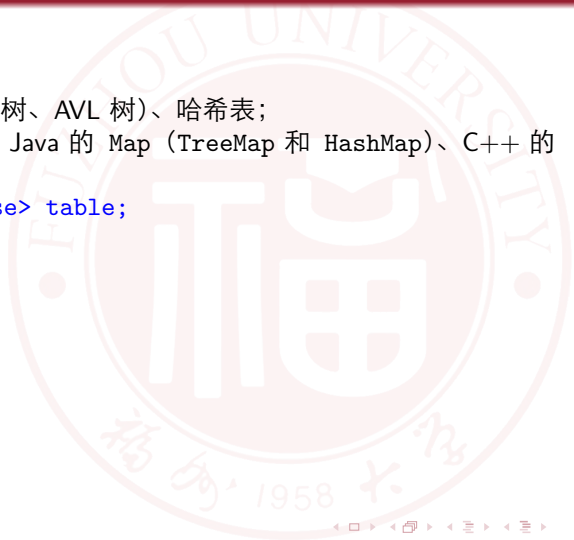
符号表的实现指导

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map（TreeMap 和 HashMap）、C++ 的 map 和 unordered_map；

```
HashMap<String, SymbolBase> table;
```

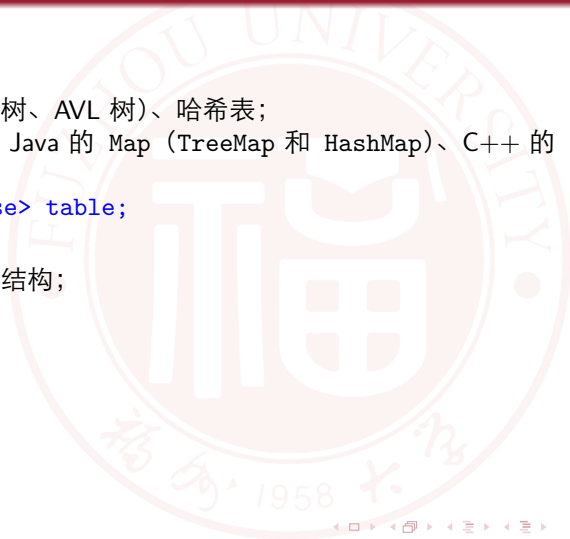
符号表的实现指导

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map（TreeMap 和 HashMap）、C++ 的 map 和 unordered_map；
`HashMap<String, SymbolBase> table;`
- 如何解析、管理作用域？



符号表的实现指导

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map（TreeMap 和 HashMap）、C++ 的 map 和 unordered_map；
`HashMap<String, SymbolBase> table;`
- 如何解析、管理作用域？
 - 理想中的方案：可持久化数据结构；



符号表的实现指导

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map（TreeMap 和 HashMap）、C++ 的 map 和 unordered_map；
`HashMap<String, SymbolBase> table;`
- 如何解析、管理作用域？
 - 理想中的方案：可持久化数据结构；
 - 写时复制（Copy-on-Write, COW）；

符号表的实现指导

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map（TreeMap 和 HashMap）、C++ 的 map 和 unordered_map；
`HashMap<String, SymbolBase> table;`
- 如何解析、管理作用域？
 - 理想中的方案：可持久化数据结构；
 - 写时复制（Copy-on-Write, COW）；
 - 可以零成本恢复历史版本；

符号表的实现指导

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map（TreeMap 和 HashMap）、C++ 的 map 和 unordered_map；
`HashMap<String, SymbolBase> table;`
- 如何解析、管理作用域？
 - 理想中的方案：可持久化数据结构；
 - 写时复制（Copy-on-Write, COW）；
 - 可以零成本恢复历史版本；
 - 用空间换时间；

符号表的实现指导

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map（TreeMap 和 HashMap）、C++ 的 map 和 unordered_map；
`HashMap<String, SymbolBase> table;`
- 如何解析、管理作用域？
 - 理想中的方案：可持久化数据结构；
 - 写时复制（Copy-on-Write, COW）；
 - 可以零成本恢复历史版本；
 - 用空间换时间；
 - 现实中的方案：

符号表的实现指导

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map（TreeMap 和 HashMap）、C++ 的 map 和 unordered_map；
`HashMap<String, SymbolBase> table;`
- 如何解析、管理作用域？
 - 理想中的方案：可持久化数据结构；
 - 写时复制（Copy-on-Write, COW）；
 - 可以零成本恢复历史版本；
 - 用空间换时间；
 - 现实中的方案：
 - 顶层浅拷贝：**注意和深度拷贝的区别**；

符号表的实现指导

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map（TreeMap 和 HashMap）、C++ 的 map 和 unordered_map；
`HashMap<String, SymbolBase> table;`
- 如何解析、管理作用域？
 - 理想中的方案：可持久化数据结构；
 - 写时复制（Copy-on-Write, COW）；
 - 可以零成本恢复历史版本；
 - 用空间换时间；
 - 现实中的方案：
 - 顶层浅拷贝：注意和深度拷贝的区别；
 - 记录新增和变化量，离开时还原；

一个反面案例

```
1 public class SymbolTable {  
2     public SymbolTable(SymbolTable other) {  
3         this.stackLayer = other.stackLayer;  
4         this.table = new HashMap<>(other.table); // 顶层浅拷贝  
5     }  
6     ...  
7 }
```

```
1 public class BuildTreeListener implements MentalListener {  
2     public void beginScope() {  
3         this.symbolTableList.add(new SymbolTable(curSymbolTable));  
4         this.curSymbolTable = this.symbolTableList.getLast();  
5         this.curSymbolTable.stackLayer++;  
6     }  
7     public void endScope() {  
8         this.symbolTableList.removeLast();  
9         this.curSymbolTable = this.symbolTableList.getLast();  
10    }  
11    ...  
12 }
```

类型检查

意义 确保程序中的操作符、表达式及赋值等行为符合语言**类型系统**的约束，避免因**类型不匹配**导致的逻辑错误或运行时崩溃。

方法 通过静态语义分析遍历抽象语法树，验证变量类型是否**兼容**（如整型与浮点型运算是否允许隐式转换），并检查函数参数类型是否与声明一致。若发现非法操作（如字符串与数值相加），编译器将抛出错误。

例子 在 C 语言中，若定义 `int a = 5; float b = 3.14;`，表达式 `a + b` 会被隐式转换为浮点型运算；但若尝试将字符串直接赋值给整型变量，类型检查会阻止此操作。

核心原因 **CPU 寄存器、内存存储时不区分数据类型。由使用数据的 ALU 负责解释数据。**

作用域分析

练习

请简述作用域和可见域的意义。



作用域分析

作用域分析：

- 意义** 确定变量、函数等标识符的**可见范围**，防止因作用域嵌套或覆盖引发的语义歧义（如局部变量屏蔽全局同名变量）。
- 方法** 基于符号表构建作用域层级结构（如树形结构），逐层解析标识符的声明位置。编译器在进入新作用域（如函数或代码块）时**创建子符号表**，退出时**销毁**，确保作用域链的正确性。
- 例子** 在 Python 中，若全局变量 $x = 10$ 与函数内局部变量 $x = 20$ 同名，函数内部访问 x 时优先引用局部变量，作用域分析确保这一逻辑的准确性。

其声明严格一致，避免因参数不匹配或非法
数量与声明是否一致，检查结构体或对象成
存在该成员、访问权限是否合规)。
void func(int a, float b)，调用时若使用
3.14)，一致性检查会分别报错“参数数量不

例子 在 C++ 中，若声明函数 `void func(int a, float b)`，调用时若使用 `func(10)` 或 `func("str", 3.14)`，一致性检查会分别报错“参数数量不足”和“类型不匹配”。

(如永远无法执行的代码块) 或非终结点), 确保程序逻辑的合理性。

分析代码块的执行路径可达性。例如, `if` 分支会被标记为不可达; 若函数返回, 后续代码是否可能被执行。

```
(true) { ... } 且后续无 break 语句, 则后续代码不可达。
```

例子 在 Java 中，若编写 `while (true) { ... }` 且后续无 `break` 语句，控制流验证会提示循环体后的代码不可达。

综合示例

```
1 x = 10  # 全局作用域
2 def foo(y):
3     x = 20      # 局部作用域覆盖全局  $x$ , 作用域分析标记局部  $x$ 
4     z = x + y   # 类型检查: 若  $y$  为字符串则报错
5     if False:  # 控制流验证: 条件恒假, 标记 if 块不可达
6         return z
7     print(z)   # 控制流验证: 此处必然执行
8 foo(5)        # 一致性检查: 参数数量正确
```



① 知识点回顾

② 语义分析

语义分析概述

符号表管理

基于 ANTLR Listener 的语义分析

③ 语法制导翻译

基于 ANTLR Listener 的语义分析

- ① 顶层元素特殊处理：不同语言对于函数、类、全局变量的声明和使用顺序要求不同，故需要不同的处理方案；
 - C/C++ 等语言强调**先声明再使用**，原则上跨模块函数调用的**声明和定义要求分开**。

基于 ANTLR Listener 的语义分析

- ① 顶层元素特殊处理：不同语言对于函数、类、全局变量的声明和使用顺序要求不同，故需要不同的处理方案；
 - C/C++ 等语言强调**先声明再使用**，原则上跨模块函数调用的**声明和定义要求分开**。因此，C/C++ 语言只需**从左至右**按顺序遍历语法树即可完成语义分析；

基于 ANTLR Listener 的语义分析

- ① 顶层元素特殊处理：不同语言对于函数、类、全局变量的声明和使用顺序要求不同，故需要不同的处理方案；
 - C/C++ 等语言强调**先声明再使用**，原则上跨模块函数调用的**声明和定义要求分开**。因此，C/C++ 语言只需**从左至右**按顺序遍历语法树即可完成语义分析；
 - Python、Java 等语言则对顶层元素或同一作用域内元素出现顺序**没有要求**，可以先使用后定义。

基于 ANTLR Listener 的语义分析

- ① 顶层元素特殊处理：不同语言对于函数、类、全局变量的声明和使用顺序要求不同，故需要不同的处理方案；
 - C/C++ 等语言强调**先声明再使用**，原则上跨模块函数调用的**声明和定义要求分开**。因此，C/C++ 语言只需**从左至右**按顺序遍历语法树即可完成语义分析；
 - Python、Java 等语言则对顶层元素或同一作用域内元素出现顺序**没有要求**，可以**先使用后定义**。这导致这些语言在顶层元素或类定义等语法结构中需要特殊处理，如先构建顶层符号表或类内成员符号表。

顶层语法定义

```
1 program: (declaration | definition | emptyStatement)*;
2 declaration: classDeclaration | functionDeclaration;
3 classDeclaration
4     : 'class' name=className? '{'
5     (variableDefinition | functionDefinition)*
6     '}' ;
7 functionDeclaration
8     : (type | 'void') Identifier '(' parametersList? ')' ';' ;
9 definition: variableDefinition | functionDefinition;
10 singleVariable: Identifier ('=' expression)?;
11 variableDefinition: type singleVariable (',' singleVariable)* ';' ;
12 functionDefinition
13     : (type | 'void') functionName=Identifier
14     '(' parametersList? ')' compoundStatement;
15 compoundStatement: '{' statement* '}' ;
```

顶层元素特殊处理示例-enterProgram-第一遍顶级作用域内符号扫描

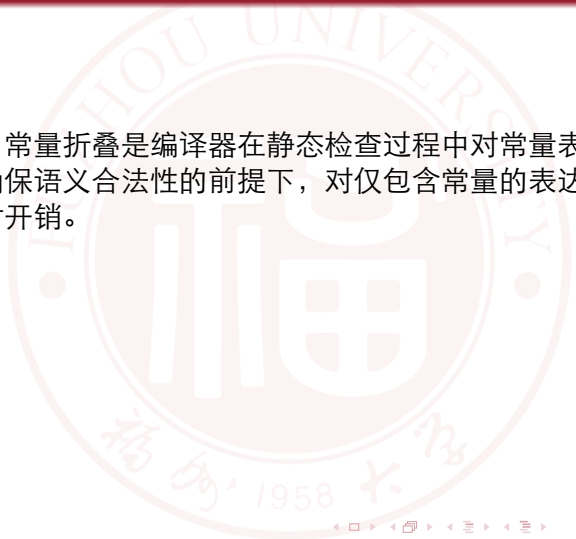
```
1 public void enterProgram(MentalParser.ProgramContext ctx) {  
2     this.tree.put(ctx, new Program());  
3     for (int i = 0, count = ctx.getChildCount(); i < count; ++i) {  
4         if (ctx.getChild(i) instanceof MentalParser.DeclarationContext declarationContext) {  
5             // 这是类声明。  
6             if (declarationContext.classDeclaration() != null) {  
7                 this.curSymbolTable.add(declarationContext.classDeclaration().className().getText(), new SymbolType());  
8             }  
9         } else if (ctx.getChild(i) instanceof MentalParser.DefinitionContext definitionContext) {  
10            // 处理变量定义和函数定义。  
11            if (definitionContext.functionDefinition() != null) {  
12                // 这是函数定义  
13                this.curSymbolTable.add(definitionContext.functionDefinition().functionName().getText(), new SymbolFunction());  
14            }  
15        }  
16    }  
17    ...  
}
```

顶层元素特殊处理示例-enterProgram 的第二次遍历

```
1  ...
2  for (int i = 0, count = ctx.getChildCount(); i < count; ++i) {
3      if (ctx.getChild(i) instanceof MentalParser.DefinitionContext definitionContext) {
4          if (definitionContext.functionDefinition() != null) {
5              MentalParser.FunctionDefinitionContext functionDefinitionContext =
6                  definitionContext.functionDefinition();
7              SymbolFunction symbolFunction =
8                  (SymbolFunction) this.curSymbolTable.getSymbol(functionDefinitionContext.functionName.getText());
9              this.existError |= symbolFunction.setFunction(this.curSymbolTable, functionDefinitionContext);
10         }
11     } else if (ctx.getChild(i) instanceof MentalParser.DeclarationContext declarationContext) {
12         if (declarationContext.classDeclaration() != null) {
13             MentalParser.ClassDeclarationContext classDeclaration = declarationContext.classDeclaration();
14             if (classDeclaration.className() != null) {
15                 SymbolType symbolType =
16                     (SymbolType) this.curSymbolTable.getSymbol(classDeclaration.className().getText());
17                 this.existError |= symbolType.setType(this.curSymbolTable, classDeclaration);
18             }
19         }
20     }
21 }
```

基于 ANTLR Listener 的语义分析

- ② 表达式的类型检查和常量折叠：常量折叠是编译器在静态检查过程中对常量表达式进行**优化**的典型场景。在确保语义合法性的前提下，对仅包含常量的表达式进行**预计算**，从而消除运行时开销。



基于 ANTLR Listener 的语义分析

- ② 表达式的类型检查和常量折叠：常量折叠是编译器在静态检查过程中对常量表达式进行**优化**的典型场景。在确保语义合法性的前提下，对仅包含常量的表达式进行**预计算**，从而消除运行时开销。

基础版 显式常量折叠，即对 `const` 修饰的变量和字面值（literals）构成的表达式进行提前计算；

基于 ANTLR Listener 的语义分析

- ② 表达式的类型检查和常量折叠：常量折叠是编译器在静态检查过程中对常量表达式进行**优化**的典型场景。在确保语义合法性的前提下，对仅包含常量的表达式进行**预计算**，从而消除运行时开销。

基础版 显式常量折叠，即对 `const` 修饰的变量和字面值 (literals) 构成的表达式进行提前计算；

进阶版 伪常量折叠，通过分析数据流图，对一些可以确定值的变量（如初始化后直接使用）与显式常量构成的表达式进行提前计算；

常量折叠-`exit``ADDITIVE_EXPRESSION`

```
1 // 先做类型检查
2 if (thisExpression.leftExpression.returnType.equals(thisExpression.rightExpression.returnType)) {
3     // 左右子表达式值类型匹配
4     if (thisExpression.op == AdditiveExpression.ADD) {
5         if (thisExpression.leftExpression.returnType.equals(SymbolTable.MENTAL_M_STRING)) {
6             // 字符串拼接场景
7             thisExpression.returnType = SymbolTable.MENTAL_M_STRING;
8             if (thisExpression.leftExpression instanceof StringLiteral) {
9                 if (thisExpression.rightExpression instanceof StringLiteral) {
10                     // 左右子节点均是常量，故创建新的语法树节点来替代当前节点
11                     StringLiteral replaceNode = new StringLiteral();
12                     replaceNode.literalContext =
13                         ((StringLiteral) thisExpression.leftExpression).literalContext.substring(0, ((StringLiteral)
14                             thisExpression.rightExpression).literalContext.substring(1);
15                     this.tree.replace(ctx, replaceNode);
16                 }
17             }
18             return;
19         } else if (thisExpression.leftExpression.returnType.equals(SymbolTable.mentalInt)) {
20             ...
```

常量折叠-整数加法

```
1  ...
2  // 左右子节点都是整数，整数加法
3  thisExpression.returnType = SymbolTable.mentalInt;
4  if (thisExpression.leftExpression instanceof IntLiteral) {
5      if (thisExpression.rightExpression instanceof IntLiteral) {
6          IntLiteral replaceNode = new IntLiteral();
7          replaceNode.literalContext =
8              ((IntLiteral) thisExpression.leftExpression).literalContext
9              + ((IntLiteral) thisExpression.rightExpression).literalContext;
10         this.tree.replace(ctx, replaceNode);
11     }
12 }
13 return;
14 }
15 } else {
16     if (thisExpression.leftExpression.returnType.equals(SymbolTable.mentalInt)) {
17         // 整数减法
18         ...
19     }
20 }
21 }
```

语法分析与语义分析的对比

	语法分析	语义分析
目标	验证代码结构是否符合语言的语法规则 (如括号匹配、语句结构顺序)	确保代码逻辑符合语言语义规则 (如类型兼容性、变量作用域)
方法	基于 CFG 构建 AST, 仅关注符号排列的合法性。	通过符号表管理和静态语义检查, 验证上下文相关约束 (如变量声明、函数参数匹配)。
处理阶段	位于词法分析之后	位于语法分析之后
输出	语法树	语法树、符号表, 甚至可以直接输出中间表示
错误的影响	无法生成可执行代码	可能生成错误代码但无法正确运行



① 知识点回顾

② 语义分析

③ 语法制导翻译

基本概念

语法制导定义

语法制导翻译 (Syntax-Directed Translation)

定义 语法制导翻译使用上下文无关文法 CFG (context free grammar) 来引导对语言的翻译，是一种面向文法的翻译技术。



语法制导翻译 (Syntax-Directed Translation)

定义 语法制导翻译使用上下文无关文法 CFG (context free grammar) 来引导对语言的翻译，是一种面向文法的翻译技术。

所处阶段 语法制导翻译横跨了**语法分析**、**语义分析**、**中间代码生成**三个阶段。

语法制导翻译 (Syntax-Directed Translation)

定义 语法制导翻译使用上下文无关文法 CFG (context free grammar) 来引导对语言的翻译，是一种面向文法的翻译技术。

所处阶段 语法制导翻译横跨了**语法分析**、**语义分析**、**中间代码生成**三个阶段。

基本思想 当归约（或推导）到某个产生式时，除了按照产生式进行相应的代换之外（语法分析），还要按照产生式所对应的语义规则执行相应的语义动作，如计算表达式的值、产生中间代码（语义分析）。

基本思想

① 如何表示语义信息？



基本思想

① 如何表示语义信息？

- 为 CFG 中的**文法符号**设置**语义属性**（如类型 T、值 val 等），用来表示语法成分对应的**语义信息**。



基本思想

- ① 如何表示语义信息？
 - 为 CFG 中的**文法符号**设置**语义属性**（如类型 T、值 val 等），用来表示语法成分对应的**语义信息**。
- ② 如何计算语义属性？



基本思想

- ① 如何表示语义信息？
 - 为 CFG 中的**文法符号**设置**语义属性**（如类型 T、值 val 等），用来表示语法成分对应的**语义信息**。
- ② 如何计算语义属性？
 - 对于给定的输入串 x ，构建 x 的**语法分析树**，并利用与产生式相关联的**语义规则**来计算分析树中各结点对应的文法符号的**语义属性值**。

基本思想

- ① 如何表示语义信息？
 - 为 CFG 中的**文法符号**设置**语义属性**（如类型 T、值 val 等），用来表示语法成分对应的**语义信息**。
- ② 如何计算语义属性？
 - 对于给定的输入串 x ，构建 x 的**语法分析树**，并利用与产生式相关联的**语义规则**来计算分析树中各结点对应的文法符号的**语义属性值**。
- ③ 对输入串的**翻译**就是根据**语义规则**进行**属性计算**。

基本思想

- ① 如何表示语义信息？
 - 为 CFG 中的**文法符号**设置**语义属性**（如类型 T、值 val 等），用来表示语法成分对应的**语义信息**。
- ② 如何计算语义属性？
 - 对于给定的输入串 x ，构建 x 的**语法分析树**，并利用与产生式相关联的**语义规则**来计算分析树中各结点对应的文法符号的**语义属性值**。
- ③ 对输入串的**翻译**就是根据**语义规则**进行**属性计算**。
- ④ **语义规则**也叫**语义子程序或语义动作**。

基本思想

- ① 如何表示语义信息？
 - 为 CFG 中的**文法符号**设置**语义属性**（如类型 T、值 val 等），用来表示语法成分对应的**语义信息**。
- ② 如何计算语义属性？
 - 对于给定的输入串 x ，构建 x 的**语法分析树**，并利用与产生式相关联的**语义规则**来计算分析树中各结点对应的文法符号的**语义属性值**。
- ③ 对输入串的**翻译**就是根据**语义规则**进行**属性计算**。
- ④ **语义规则**也叫**语义子程序或语义动作**。
- ⑤ 将语义规则同语法规则（产生式）联系起来有两种方式：
 - 语法制导定义（Syntax-Directed Definitions, SDD）
 - 语法制导翻译方案（Syntax-Directed Translation Scheme, SDT）

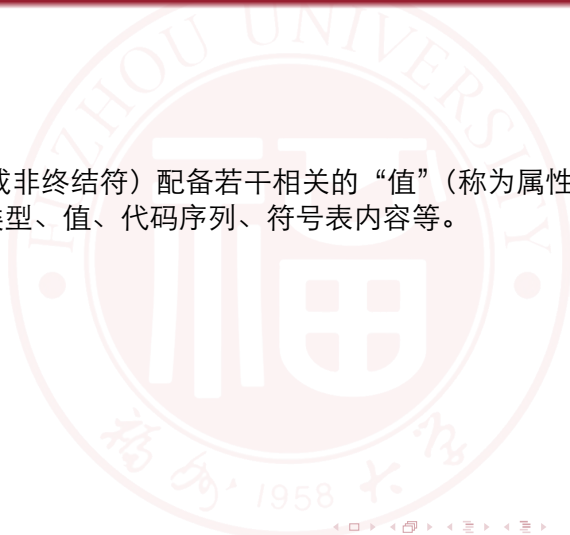
- ① 知识点回顾
- ② 语义分析
- ③ 语法制导翻译
基本概念
语法制导定义

- SDD 是对 CFG 的推广

- 授课教师：柯嵩宇
编译原理-语义分析 01

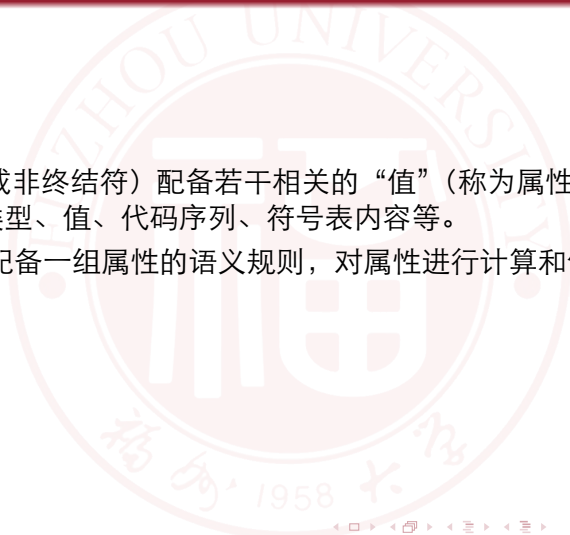
语法制导定义 (Syntax-Directed Definitions)

- SDD 是对 CFG 的推广
- SDD 为每个文法符号（终结符或非终结符）配备若干相关的“值”（称为属性），代表与文法符号相关信息，如类型、值、代码序列、符号表内容等。



语法制导定义 (Syntax-Directed Definitions)

- SDD 是对 CFG 的推广
- SDD 为每个文法符号（终结符或非终结符）配备若干相关的“值”（称为属性），代表与文法符号相关信息，如类型、值、代码序列、符号表内容等。
- SDD 为每个文法的每个产生式配备一组属性的语义规则，对属性进行计算和传递。



语法制导定义 (Syntax-Directed Definitions)

- SDD 是对 CFG 的推广
- SDD 为每个文法符号（终结符或非终结符）配备若干相关的“值”（称为属性），代表与文法符号相关信息，如类型、值、代码序列、符号表内容等。
- SDD 为每个文法的每个产生式配备一组属性的语义规则，对属性进行计算和传递。
- 如果 X 是一个**文法符号**， a 是 X 的一个**属性**，则用 $X.a$ 表示**属性** a 在某个标号为 X 的**分析树结点**上的**值**。

例 (SDD 示例)

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

- 文法符号: L 、 E 、 T 、 F 、 $digit$
- 属性: val
- 语义规则: 通常写成表达式的形式, 用于计算文法符号的属性值

例 (SDD 示例)

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

- 文法符号: L 、 E 、 T 、 F 、 $digit$
- 属性: val
- 语义规则: 通常写成表达式的形式, 用于计算文法符号的属性值

文法符号的属性分类:

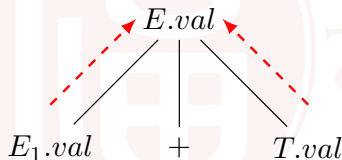
- 综合属性 (Synthesized Attribute)
- 继承属性 (Inherited Attribute)

综合属性

- 在分析树结点 N 上的非终结符 A 的综合属性只能通过 N 的**子结点**或 **N 本身**的属性值来定义。

例

产生式	语义规则
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$



- 终结符可以具有综合属性。终结符的综合属性值是由词法分析器提供的词法值，在 SDD 中没有计算终结符属性值的语义规则。
 - token: < 种类值, 属性值 (词法值) >

- 在分析树结点 N 上的非终结符 A 的**继承属性**只能通过**N 的父结点**、**N 的兄弟结点**或 **N 本身**的属性值来定义。

A diagram showing a node D at the top. Two solid black lines branch down from D to $T.type$ on the left and $L.inh$ on the right. A dashed red arrow points from $T.type$ to $L.inh$.

- 38 / 41

SDD 综合示例

例 (分析 $3 * 5 + 4n$ 的过程)

产生式	语义规则
$L \rightarrow E n$	<i>print</i> (<i>E.val</i>) ¹
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$ ²

SDD 综合示例

例 (分析 $3 * 5 + 4n$ 的过程)

产生式	语义规则
$L \rightarrow E n$	<i>print</i> (<i>E.val</i>) ¹
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$ ²

¹虚拟综合属性、副作用

Diagram illustrating the parse tree for the expression $(3 * 5) + 4$ using the LR(0) items and transitions:

```
graph TD
    L --> E
    L --> n
    E --> E1[E]
    E --> plus["+"]
    E --> T1[T]
    E1 --> T2[T]
    E1 --> star["*"]
    E1 --> F1[F]
    T2 --> F2[F]
    T2 --> 3["3"]
    F1 --> 5["5"]
    T1 --> F3[F]
    T1 --> 4["4"]
    F3 --> 3
```

The tree structure shows the derivation of the expression $(3 * 5) + 4$ from the start symbol L . The root L expands to E and n . The non-terminal E expands to E , $+$, and T . The inner E expands to T , $*$, and F . The T node expands to F and the terminal 3 . The F node expands to the terminal 5 . The outer T node expands to F and the terminal 4 . The F node expands to the terminal 3 .

```

graph TD
    L --> E1[E]
    L --> n[n]
    E1 --> E2[E]
    E1 --> plus[+]
    E1 --> T1[T]
    E2 --> T2[T]
    E2 --> star[*]
    E2 --> F1[F]
    T2 --> F2[F]
    F2 --> 3[3]
    F1 --> 5[5]
    T1 --> F3[F]
    F3 --> 4[4]
  
```

Diagram illustrating the evaluation of the expression $3 * 5 + 4$ using a parse tree structure. The root node is $L.val = 19$. The tree structure shows the evaluation of the expression, with intermediate values stored in nodes.

```

graph TD
    L["L.val = 19"] --- E1["E.val = 19"]
    L --- n["n"]
    E1 --- E15["E.val = 15"]
    E1 --- plus["+"]
    E1 --- T4["T.val = 4"]
    E15 --- T15["T.val = 15"]
    T15 --- T3["T.val = 3"]
    T15 --- star["*"]
    T15 --- F5["F.val = 5"]
    T3 --- F3["F.val = 3"]
    F3 --- digit3["digit.lexval = 3"]
    F5 --- digit5["digit.lexval = 5"]
    T4 --- F4["F.val = 4"]
    F4 --- digit4["digit.lexval = 4"]
  
```

The diagram shows the following structure:

- Root: $L.val = 19$
 - Left child: $E.val = 19$
 - Left child: $E.val = 15$
 - Left child: $T.val = 15$
 - Left child: $T.val = 3$
 - Left child: $F.val = 3$
 - Left child: $digit.lexval = 3$
 - Right child: $*$
 - Right child: $F.val = 5$
 - Left child: $digit.lexval = 5$
 - Right child: $+$
 - Right child: $T.val = 4$
 - Left child: $F.val = 4$
 - Left child: $digit.lexval = 4$

Diagram illustrating the evaluation of the expression $3 * 5 + 4$ using a parse tree structure. The root node is $L.val = 19$. The tree structure shows the evaluation of the expression, with intermediate values stored in nodes.

```

graph TD
    L["L.val = 19"] --- E1["E.val = 19"]
    L --- n["n"]
    E1 --- E15["E.val = 15"]
    E1 --- plus["+"]
    E1 --- T4["T.val = 4"]
    E15 --- T15["T.val = 15"]
    T15 --- T3["T.val = 3"]
    T15 --- star["*"]
    T15 --- F5["F.val = 5"]
    T3 --- F3["F.val = 3"]
    F3 --- digit3["digit.lexval = 3"]
    F5 --- digit5["digit.lexval = 5"]
    T4 --- F4["F.val = 4"]
    F4 --- digit4["digit.lexval = 4"]
  
```

The diagram shows the following structure:

- Root: $L.val = 19$
 - Left child: $E.val = 19$
 - Left child: $E.val = 15$
 - Left child: $T.val = 15$
 - Left child: $T.val = 3$
 - Left child: $F.val = 3$
 - Left child: $digit.lexval = 3$
 - Right child: $*$
 - Right child: $F.val = 5$
 - Left child: $digit.lexval = 5$
 - Right child: $+$
 - Right child: $T.val = 4$
 - Left child: $F.val = 4$
 - Left child: $digit.lexval = 4$